

SC2001 / ALGORITHM DESIGN AND ANALYSIS

Dijkstra's Algorithm

LAB 2

Gupta Dhruv
Gajulapalli Anish Reddy
Koh Jin En

SC2001 • PROJECT 2



The question

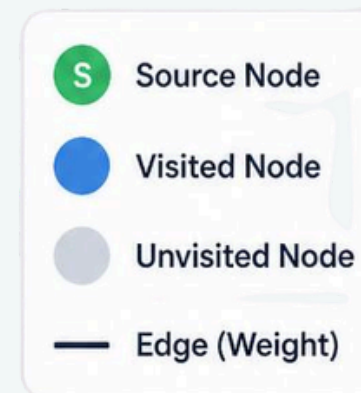
**Given one source.
Find shortest
distance to every
other vertex.**

Dijkstra's algorithm solves this for graphs with edge weights ≥ 0 .

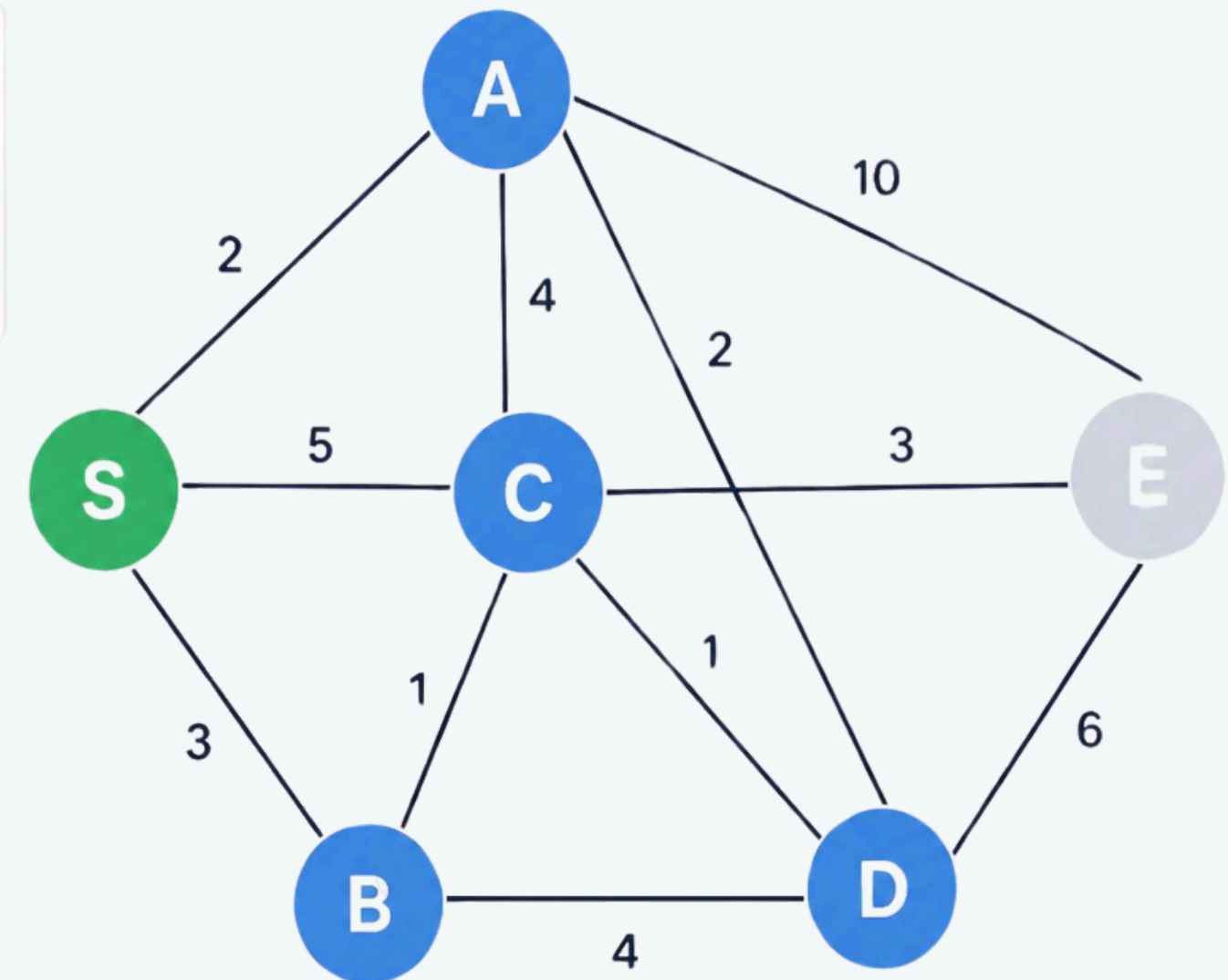
However, its run time depends on how we store the graph information and select the next vertex to visit.

We go over 2 of its implementations and compare them to each other.

Dijkstra's Algorithm



Shortest Distance from Source (S)	
S	0
A	2
B	3
C	6
D	7
E	∞



Matrix + array / data structures

V×V Adjacency matrix

Each cell contains an edge weight representing distance between the 2 vertices,
“None” means there is no edge,
which allows “0” to remain a valid weight.

Priority queue

distances[] + visited[]

1. distances: which stores the best known distances upto the current vertex. Starts by taking INFINITY distance to every node.
2. visited: which tracks vertices already visited.

To go to the next nearest unvisited vertex,
we scan the entire distances array.
Each selection therefore takes $O(V)$ time.

```
def find_nearest_unvisited_vertex(distances, visited):  
    smallest_distance = INFINITY  
    nearest_vertex = -1  
  
    for vertex in range(len(distances)):  
        if (not visited[vertex] and  
            distances[vertex] < smallest_distance):  
            smallest_distance = distances[vertex]  
            nearest_vertex = vertex  
  
    return nearest_vertex    #  $O(V)$  scan every call
```

Matrix + array / the main loop

Each iteration selects the nearest unvisited vertex and marks it as visited.

In each iteration, we scan the current node's entire matrix row to find neighbours.

For each unvisited neighbour, we check whether travelling through the current vertex gives a shorter distance. If it does, we update the distance and the parent node.

```
for step in range(number_of_vertices):
    current_vertex = find_nearest_unvisited_vertex(distances, visited)
    if current_vertex == -1:
        break # rest are unreachable
    visited[current_vertex] = True

    for neighbour in range(number_of_vertices): # whole row: O(V)
        edge_weight = adjacency_matrix[current_vertex][neighbour]
        if edge_weight is None or visited[neighbour]:
            continue
        new_distance = distances[current_vertex] + edge_weight
        if new_distance < distances[neighbour]:
            distances[neighbour] = new_distance # O(1) update
            previous[neighbour] = current_vertex
```

Matrix + array / complexity

There are at most V iterations.
Each performs an $O(V)$ minimum selection, and an $O(V)$ row scan.
Multiplying these gives $O(V^2)$ overall.
The adjacency matrix also requires $O(V^2)$ space.

Work per Iteration

Choosing nearest unvisited vertex: $O(V)$

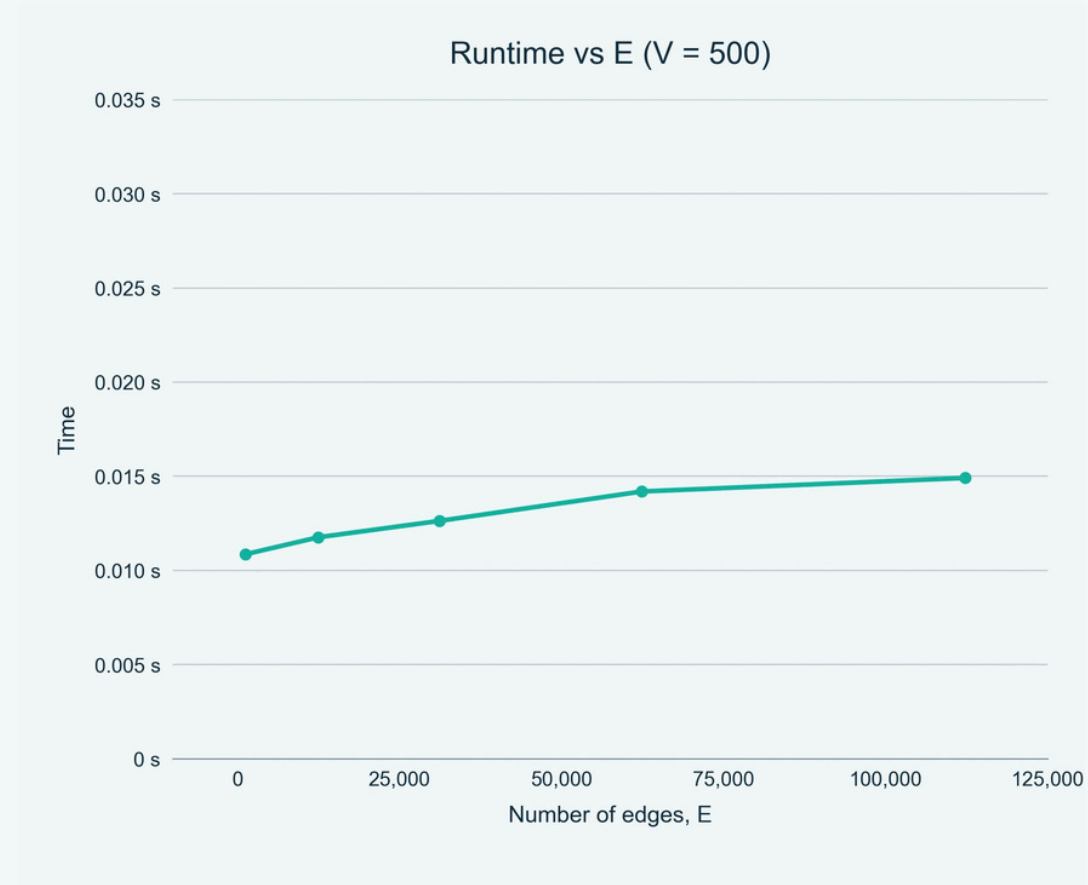
Scan its matrix row: $O(V)$

Total per iteration: $[O(V) + O(V)]$

At most V iterations

Therefore, total runtime:
 $V \times [O(V) + O(V)] = O(V^2)$

Matrix + array / empirical results



Our tests support quadratic growth.
As V increases, time by V squared stays close to five x 10⁻⁸

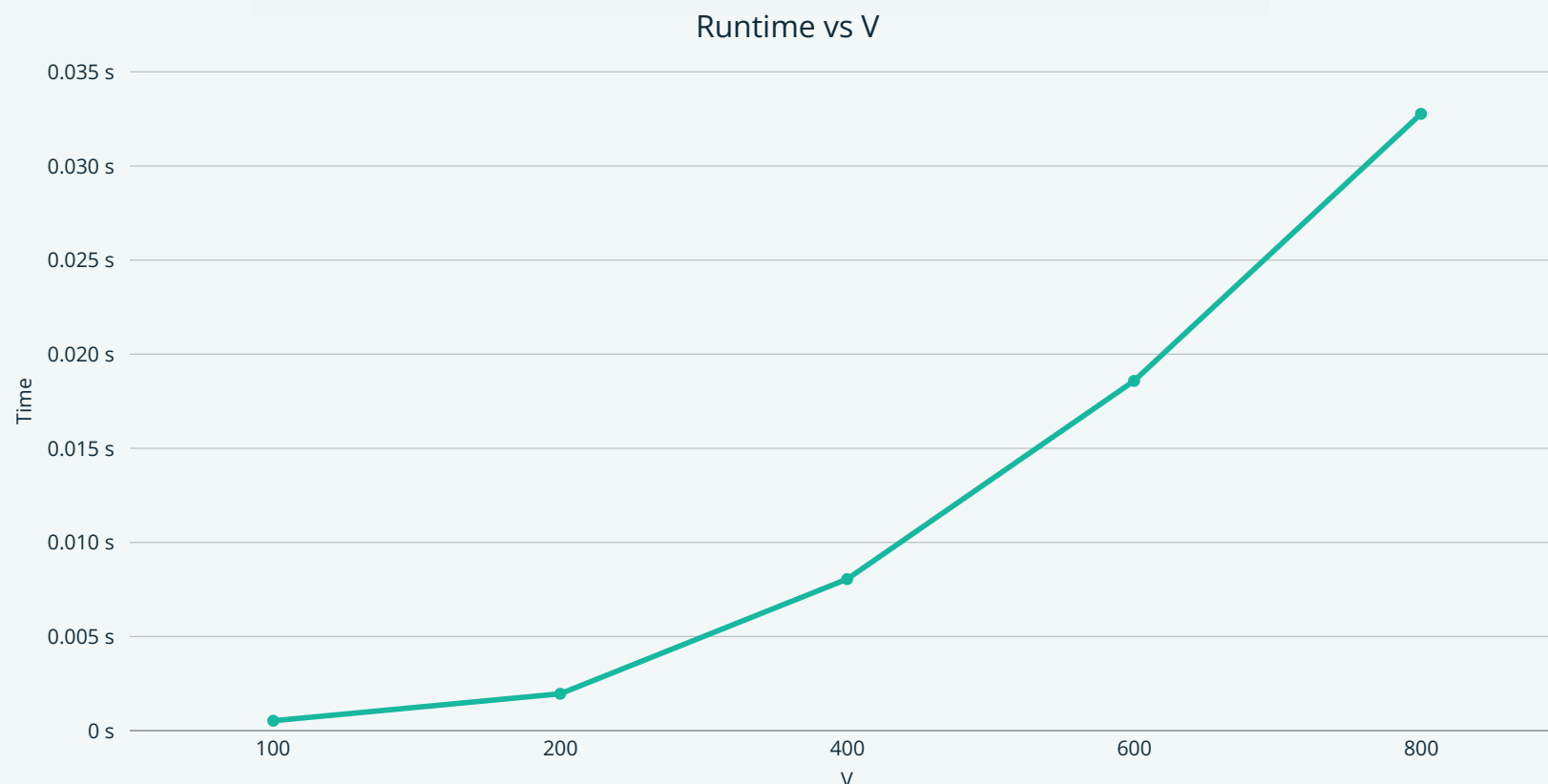
$$\approx 5 \times 10^{-8}$$

Time / V²

With V fixed at 500, increasing the number of edges by approximately 90 times increases runtime by only 1.37 times.
This shows that vertex count dominates this implementation's runtime, The O(V²) time complexity is independent of E. Actual runtime can still increase slightly because more edges require distance checks and updates

FIXED V = 500

90× edges →
only 1.37× time



Lists + min-heap / data structures

adjacency_list[u]

Stores only real (v, weight) pairs.

Binary MinHeap + position[]

position[v] gives v's heap index in $O(1)$.

Heap operation	Cost
insert	$O(\log V)$
extract_min	$O(\log V)$
decrease_key	$O(\log V)$
contains	$O(1)$

Lists + min-heap / the main loop

```
while not heap.is_empty():
    current_vertex, current_distance = heap.extract_min()
    if current_distance == INFINITY:
        break                # rest are unreachable

    for neighbour, edge_weight in adjacency_list[current_vertex]:
        if not heap.contains(neighbour):
            continue          # distance already final
        new_distance = current_distance + edge_weight
        if new_distance < distances[neighbour]:
            distances[neighbour] = new_distance
            previous[neighbour] = current_vertex
            heap.decrease_key(neighbour, new_distance)
```

1×

each vertex extracted

Only real edges are scanned.

decrease_key / the position array

```
def decrease_key(self, vertex, new_key):
    index = self.position[vertex]      # 0(1) lookup
    self.keys[index] = new_key
    self.sift_up(index)                # 0(log V)

def sift_up(self, index):
    while index > 0:
        parent = (index - 1) // 2
        if self.keys[parent] <= self.keys[index]:
            break
        self.swap(index, parent)      # also updates position[]
        index = parent
```

Without position[]

$O(V)$

With position[]

$O(1)$ lookup

Lists + min-heap / theoretical complexity

V extracts

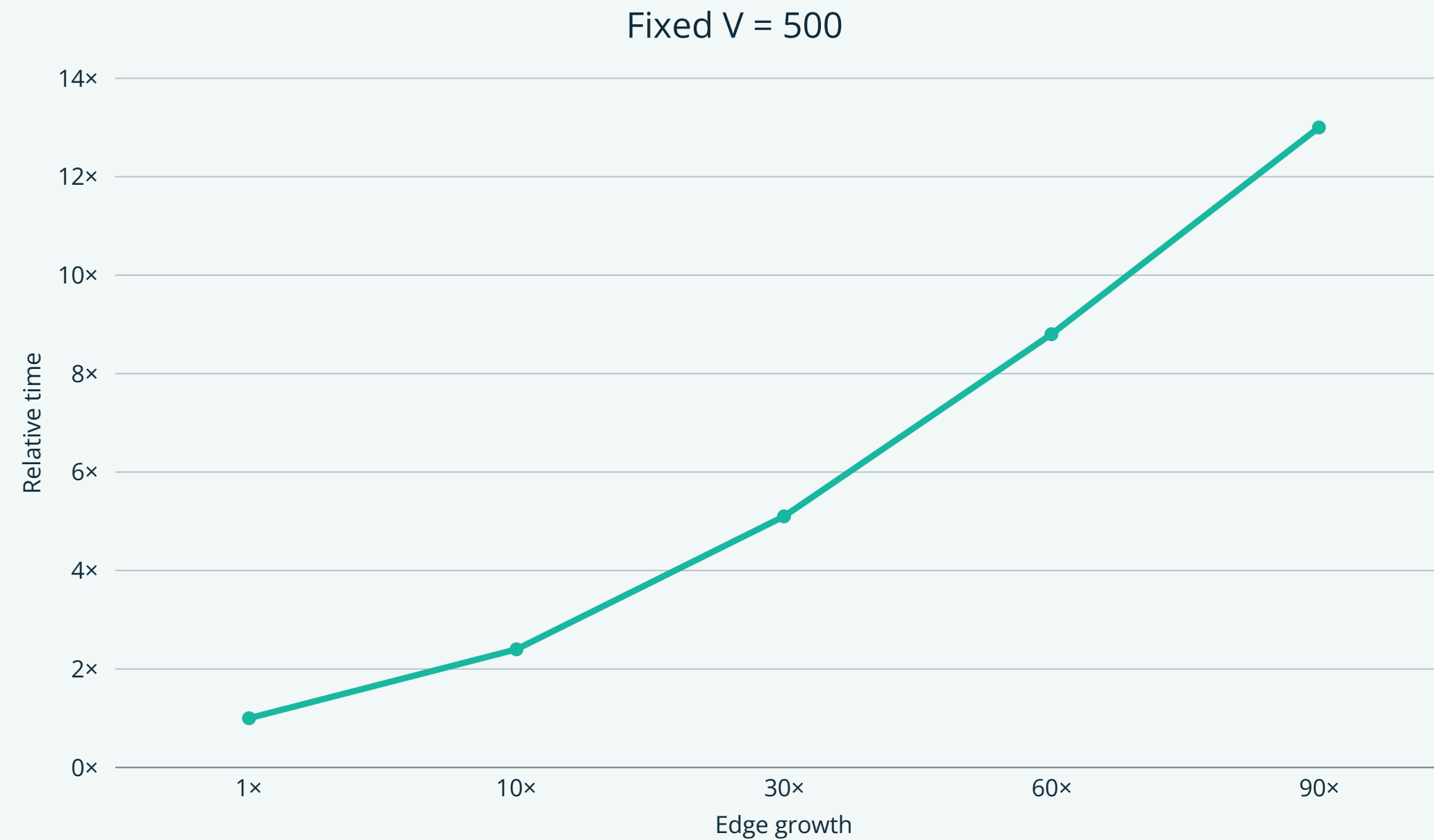
$V \log V$

E relaxations

$E \log V$

$O((V + E) \log V)$

Lists + min-heap / empirical results



90× edges
→ **13× time**

$V = 50,000$

0.90 s

when $E = 5V$

Theory comparison

operation	part (a): adjacency matrix and array priority queue	part(b): adjacency list and min heap priority queue
Initialisation	$O(V)$	$O(V)$
Extract-Min (per call)	$O(V)$ (scan all unvisited elemets)	$O(\log V)$ (sift-down binary heap)
Extract-Min (total for all vertices)	$O(V ^2)$	$O(V \log V)$
Scan for neighbours	$O(V ^2)$ (inspects all $ V $ cells per vertex)	$O(E)$ (traverses active incident edges)
Distance Update (per call)	$O(1)$ (assign array index directly)	$O(\log V)$ (sift-up using position tracking)
Distance Update (worst-case total)	$O(E)$	$O(E \log V)$
Total Worst-Case Time Complexity	$O(V ^2)$	$O((V + E) \log V)$
Typical / Empirical Time Complexity	$\sim O(V ^2)$ (independent of $ E $)	$\sim O(E + V \log V)$ (only few edges sift up)
Space Complexity	$O(V ^2)$	$O(V + E)$

fixed |V| at 500, |E| increases

Density	E	Part (a) time (s)	Part (b) time (s)	Faster
1%	1,247	0.01237	0.00124	(b), ~10x faster
10%	12,475	0.01320	0.00493	(b), ~2.7x faster
25%	31,187	0.01467	0.01313	(b), ~1.1x faster
50%	62,375	0.01820	0.02153	(a), ~1.2x faster
75%	93,562	0.01919	0.02424	(a), ~1.3x faster
90%	112,275	0.01998	0.02490	(a), ~1.2x faster
100%	124,750	0.02104	0.02580	(a), ~1.2x faster

Both parts run on the same laptop, on identical graphs (average of 5 runs). The lines cross at about 33% density.

fixed density ~25%, |V| increases

V	E (~25%)	Part (a) time (s)	Part (b) time (s)	Faster
100	1,237	0.000576	0.000320	(b), ~1.8x faster
200	4,975	0.002259	0.000974	(b), ~2.3x faster
400	19,950	0.009648	0.005830	(b), ~1.7x faster
600	44,925	0.022774	0.017839	(b), ~1.3x faster
800	79,900	0.038653	0.033799	(b), ~1.1x faster

Same laptop, same graphs. At 25% density (b) stays ahead, but the gap narrows as V grows.

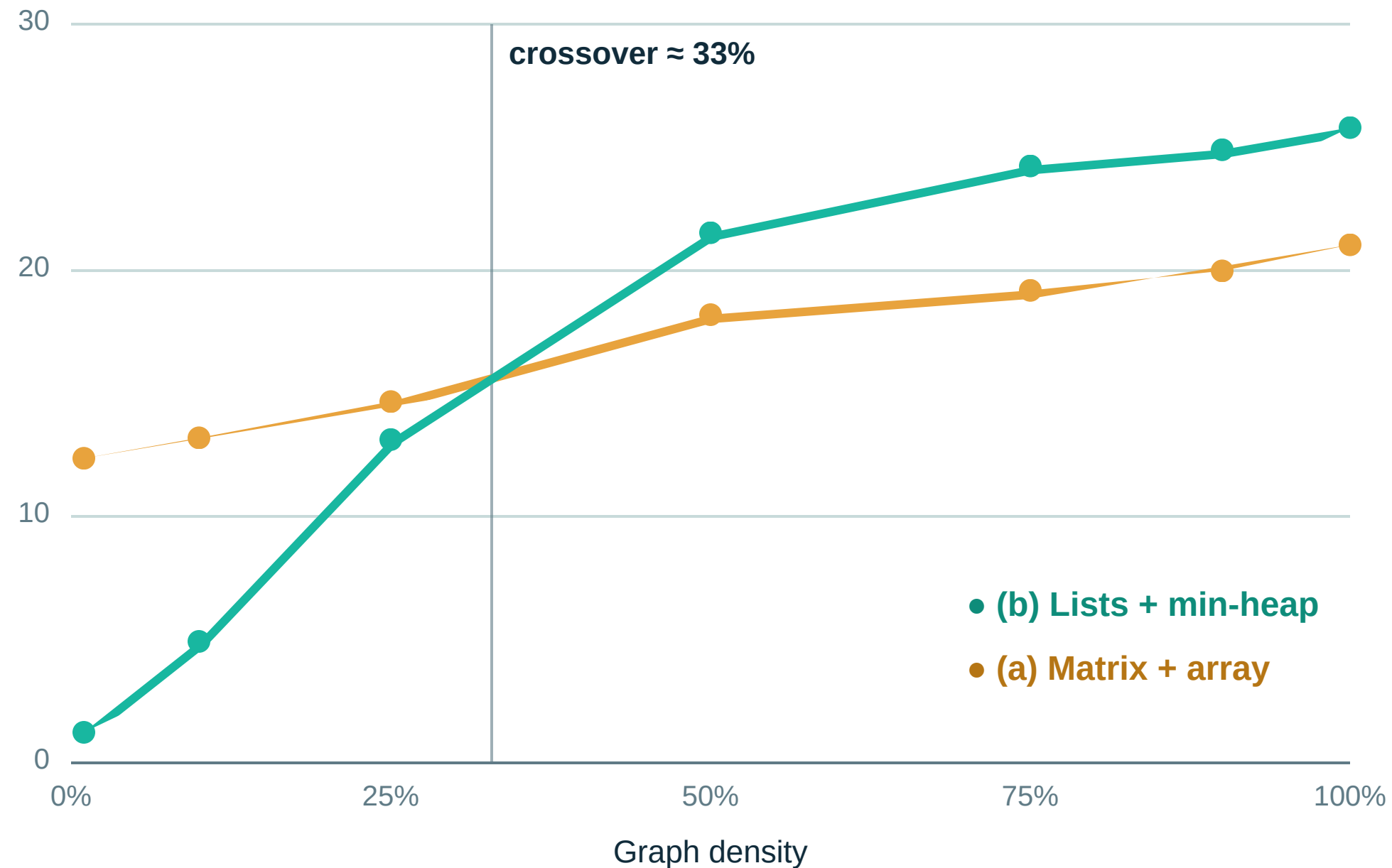
Memory / the hard limit

V (E = 5V)	Matrix entries (part a)	List entries (part b)
500	$500^2 = 250,000$	$500 + 2(2500) = 5,500$
5,000	$5000^2 = 25,000,000$	$5000 + 2(25\ 000) = 55,000$
50,000	$50000^2 = 2,500,000,000$	$50000 + 2(250000) = 550,000$

At $V = 500 \rightarrow$ both run easily
At $|V| = 5000 \rightarrow$ both run
At $|V| = 50000 \rightarrow$ matrix allocation fails but list completes in 0.90s

How Code Simplicity and Constant Costs Affect Measured Results

Time (ms), $V = 500$, same laptop



part a

**contiguous array
and low constant
factor**

VS

part b

**list traversal and
high constant factor**

Which one should you use?

MATRIX + ARRAY

Dense graphs

On the same laptop, (a) wins once density passes about 33% (around $1.2\text{--}1.3\times$ faster at $|V| = 500$).

For dense graphs ($|E| \approx |V|^2$), (a) only does $O(|V|^2)$ cheap array checks, while (b) traverses $O(|E|)$ list entries plus heap work, worst case $O(|V|^2 \log |V|)$.

LISTS + HEAP

Large or sparse graphs

For large graphs ($|V| \geq 1000$ up to $|V| = 50,000$ and beyond), Part (b) scales linearly to $|V| = 50,000$ (0.90s), whereas Part (a) becomes impractical past $|V| \approx 1,000$ due to quadratic memory growth $\Theta(|V|^2)$.

For sparse graphs ($|E| \ll |V|$), When vertices have a limited degree (for eg. $|E| \approx O(|V|)$), Part (b) $O(|E| \log |V|)$ complexity completely outperforms the $\Theta(|V|^2)$ matrix scan.

Thank you.

Gupta Dhruv

Gajulapalli Anish Reddy

Koh Jin En